CHIMERA, explained from zero

A complete guide to the project for people who have never seen it, and preparation for a viva

Adithyan C S S Ajay M Rohith Pranov V
Computer Science and Engineering, Vellore Institute of Technology, Vellore

October 2026

How to read this guide

Part 1 (Sections 1–4) assumes nothing: it explains what the cloud, Kubernetes and autoscaling are, and what problem the project solves. Part 2 (Sections 5–8) explains how CHIMERA works and how it was tested. Part 3 (Sections 9–12) gives the results, including where CHIMERA does *not* win, and what went wrong along the way. Part 4 (Sections 13–16) is for the viva: the project map, a demo guide, about sixty questions with answers, a glossary, and a one-page sheet of numbers to memorise.
If you have ten minutes, read Section 4 (the one-page summary) and Section 16 (the numbers). If you have an hour, read everything.

Contents

Part 1: The basics **3**

1 The world this project lives in **3**
1.1 Applications run on servers, and servers cost money 3
1.2 Containers, pods and Kubernetes 3
1.3 Two ways to give an application more power 3
1.4 Autoscaling 4

2 Words you must know **4**

3 The problem CHIMERA solves **5**

4 The whole project on one page **5**

Part 2: How CHIMERA works and how it was tested **7**

5 How CHIMERA works, step by step **7**

6 The competitors (baselines) **8**

7 How it was tested **9**
7.1 The simulator . . . 9
7.2 The traffic (four sets of traces) . . . 9
7.3 The fair comparison: at equal CPU . . . 9
7.4 Statistics . . . 9
7.5 Scale of the experiments . . . 9

Part 3: Results **10**

8 Where CHIMERA outperforms **10**
8.1 At equal CPU, against everything . . . 10
8.2 At default settings . . . 10
8.3 It still holds when the simulator's rules change . . . 11
8.4 Which parts matter (ablation) . . . 11

9 On a real cluster **12**
9.1 It works on real Kubernetes . . . 12
9.2 Head-to-head with the Kubernetes HPA . . . 12

10 Where CHIMERA does *not* win **13**

11 The story of the project: versions, mistakes and fixes **13**
11.1 Versions . . . 13
11.2 Mistakes found and corrected (this is a strength, explain it as one) . . . 13

12 What is new, and what is not **14**

13 Limitations **14**

Part 4: Viva preparation **15**

14 Project map and how to run it **15**
14.1 Important files . . . 15
14.2 Live demo in two steps . . . 15

15 Viva questions and answers **15**

16 Glossary **19**

17 One page of numbers to remember **20**

Part 1 The basics

1 The world this project lives in

1.1 Applications run on servers, and servers cost money

When you open an app, your phone sends a *request* to a program running on a computer in a data centre. That program does some work and sends a *response*. The computer's processor (CPU) is what does the work. CPU power is measured in **cores**: one core can run one stream of work at a time. Companies rent cores from cloud providers (Amazon, Google, Microsoft, Huawei, Alibaba) and pay for every core, for every minute, whether it is busy or idle.

So there are two things a company wants at the same time:

1. **Fast responses.** Users leave if the app is slow.
2. **Low cost.** Every idle core is money thrown away.

These pull in opposite directions. Many cores: fast but expensive. Few cores: cheap but slow when traffic rises. Traffic also changes all the time: quiet at night, busy at noon, and sometimes a sudden spike (a sale, a news event).

1.2 Containers, pods and Kubernetes

Modern applications are packaged as **containers**: a program plus everything it needs, which runs the same way on any machine. **Kubernetes** is the most widely used system for running containers on a group of machines (a **cluster**). In Kubernetes:

- A **pod** is one running copy of the application (one or more containers).
- A **replica** is another identical copy of the pod. Ten replicas means ten copies sharing the traffic.
- Each pod has a **CPU limit**: the maximum CPU it may use, for example `500m` ("500 millicores", half a core) or `2` (two cores).

1.3 Two ways to give an application more power

	Horizontal scaling	**Vertical scaling**
What changes	the number of pods (replicas)	the CPU of each pod
Analogy	open more checkout counters in a shop	make each cashier work faster
Speed	**slow**: a new pod must be scheduled, started and become ready, typically tens of seconds	**fast** since Kubernetes 1.33: about 0.1 s, without restarting the pod ("in-place resize")
Kubernetes tool	HPA (Horizontal Pod Autoscaler)	VPA (Vertical Pod Autoscaler)

Until recently, changing a pod's CPU meant killing the pod and starting a new one, which was slow and disruptive, so vertical scaling was rarely used for fast reactions. **In-place pod resize** (Kubernetes Enhancement Proposal 1287, on by default from Kubernetes 1.33 in 2025, stable in 1.35) changed this: the CPU of a running pod can now be changed in a fraction of a second, without a restart. This is the key fact behind the whole project.

1.4 Autoscaling

An **autoscaler** is a program that watches the application and changes its resources automatically. The two standard ones in Kubernetes:

- **HPA** looks at average CPU utilisation. If it is above a target (say 70 %), it adds pods; if below, it removes pods.
- **VPA** looks at past CPU usage and sets a sensible CPU size for each pod.

Run together on the same signal, they can fight: the HPA adds pods, so each pod looks less busy, so the VPA shrinks the pods, so each pod looks busy again, so the HPA adds pods... And neither of them knows that a pod it just ordered is not serving yet.

2 Words you must know

Latency (response time). How long one request takes, in seconds.

SLO (service level objective). A promise about latency, for example "every request should finish within 0.5 s". A request that takes longer is an **SLO violation**. The main quality number in this project is the **SLO violation rate**: the percentage of requests that were too slow. Lower is better.

p99 latency. Sort all response times; the p99 is the value 99 % of requests are faster than. It describes the slowest experiences (the "tail").

Utilisation. How busy the CPU is: demand divided by capacity. At 50 % utilisation the CPU is busy half the time.

Why high utilisation is dangerous. When a CPU is nearly full, requests start queueing behind each other, and waiting time grows very fast, not gradually. A simple queueing rule used in this project: response time $\approx$ service time $\times \frac{1}{1-u}$, where u is utilisation. At 50 % that is 2 times the service time; at 90 % it is 10 times; at 95 % it is 20 times. This is why autoscalers keep a target below 100 %.

Core-minutes (CPU cost). Cores allocated multiplied by minutes. Two cores for 30 minutes is 60 core-minutes. This is the project's cost measure. (Pod-minutes are misleading because pods can have different sizes.)

Trace. A recording of real traffic: how many requests arrived each minute. Tests replay traces so every controller faces exactly the same traffic.

Forecast. A prediction of next minute's traffic.

Uncertainty interval. Instead of "next minute will have 100 requests", a range: "between 80 and 130, with 90 % confidence".

Conformal prediction. A statistical method that turns any forecaster into one with honest intervals: it looks at how wrong the forecaster has been recently and widens the interval just enough to cover the truth 90 % of the time. **CQR** (conformalized quantile regression) is one version of it.

Control theory. The engineering of feedback systems (thermostats, cruise control, autopilots): measure, compare with a target, correct, repeat.

LQR (linear-quadratic regulator). A classic, mathematically optimal way to choose corrections: it balances "how far are we from the target" against "how big is the correction". The answer is a fixed matrix K; the correction is $u = -Ke$, where e is the error. K is found by solving the **Riccati equation**.

Lyapunov function. A mathematical "energy" that always decreases if a system is stable. If you can show one exists, the system cannot spiral out of control.

Baseline. An existing method you compare against.

Ablation. Removing one part of your method to see how much that part matters.

Simulator. A program that imitates a cluster so thousands of experiments can run quickly and repeatably.

Seed. The starting number for random choices in a simulation. Same seed, same random request arrivals: a fair comparison.

3 The problem CHIMERA solves

The problem in one sentence

Kubernetes now has a **fast** way to add capacity (resize a pod's CPU in 0.1 s) and a **slow** way (add a pod, tens of seconds). How should one controller use both together to keep requests fast at the lowest CPU cost?

Why the existing tools fall short:

1. **HPA** only adds pods. When traffic jumps, new pods need tens of seconds, and requests are slow in the meantime. To be safe it must keep spare pods running all the time, which costs money.
2. **VPA** only resizes, from history, and is not designed to react within seconds.
3. **HPA and VPA together** interfere, as described above.
4. **Predictive scalers** (AAPA, MagicScaler) forecast traffic, which helps, but still use only the slow actuator.
5. **KOSMOS**, the closest existing system, uses both, but as two separately designed loops running at very different speeds (5 s and 300 s).
6. **Nobody models "pods in flight".** If an autoscaler orders 2 new pods and checks again 15 seconds later, before they are ready, it may order 2 more for the same need.

4 The whole project on one page

CHIMERA in five sentences

What: CHIMERA is an autoscaler for Kubernetes that controls both the number of pods and the CPU per pod with one coordinated decision. **How:** a fast loop (every 5 s) resizes CPU in place the moment a surge appears and gives CPU back as soon as it passes; a slow loop (every 15 s) adds or removes pods only when CPU resizing cannot absorb the change, and it remembers pods that are still starting. **How much:** it decides the total capacity by weighing the expected number of slow requests against the price of extra CPU, using honest forecast ranges (conformal prediction). **Evidence:** in simulation, at the same CPU cost, it has fewer SLO violations than six other autoscalers on 47 of 48 comparisons on a held-out set and on all 117 on twenty real traces it was never designed on; on a real Kubernetes cluster against the standard HPA, 7 of 9 HPA runs were beaten outright and no CHIMERA run was. **Honesty:** its advantage is a few percentage points, not dramatic; some ideas it started with turned out to do nothing; the real-cluster study is small.

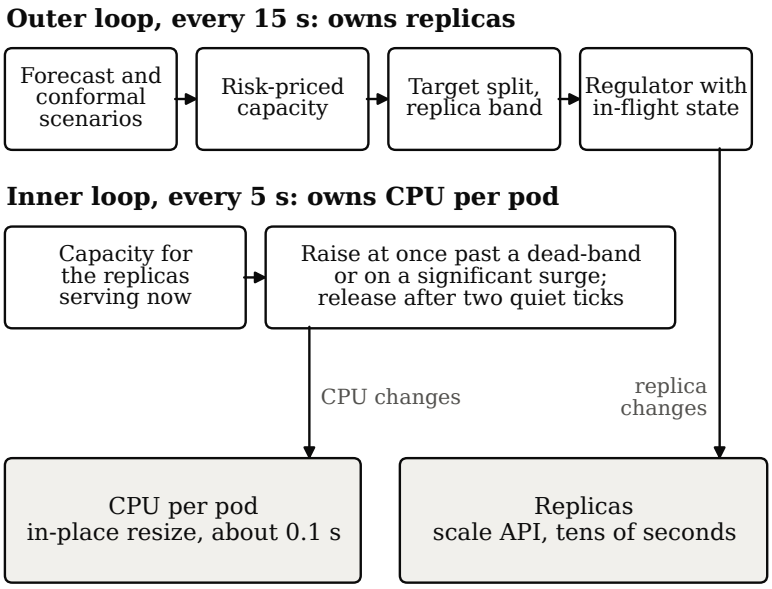


Figure 1. The two loops. The outer loop (top) owns the number of pods; the inner loop (middle) owns the CPU of each pod.

Part 2 How CHIMERA works and how it was tested

5 How CHIMERA works, step by step

Every 15 seconds the **outer loop** runs steps 1–4. Every 5 seconds the **inner loop** runs step 5.

Step 1: Forecast the traffic, with an honest range

A machine-learning model (LightGBM quantile regression) predicts next minute's traffic as a range (CQR, 90 % confidence). Because a learned model can be badly wrong on traffic it has never seen, a second, simple margin is kept beside it: "last value plus the size of recent minute-to-minute jumps". A drift detector (ADWIN) notices when the traffic pattern has changed and makes the margin forget the old pattern.

Step 2: Decide how much total CPU is worth paying for (risk pricing)

Instead of the usual fixed rule "keep utilisation at 70 %", CHIMERA asks: *for each extra core, how many slow requests does it prevent, and is that worth its price?* It takes several possible futures (current traffic, current traffic plus each recent jump, the forecast's upper bound), and picks the capacity c that minimises

$$\underbrace{\text{expected number of slow requests over those futures}}_{\text{bad for users}} + \underbrace{\lambda \times c}_{\text{cost of CPU}} .$$

The price λ is set so that if the future were certain, the answer is simply "demand divided by a target utilisation" (CHIMERA's default setting is 0.46). When the future is uncertain, it buys extra headroom only where the risk is real.

Step 3: Split the capacity into pods × CPU, and keep pods still

Total capacity c can be reached many ways: 2 pods of 2 cores, 4 pods of 1 core, and so on. Adding a pod costs boot time (a pod that is starting is paid for but does no work). So CHIMERA uses a **replica band**: as long as the current number of pods can carry the needed capacity by resizing CPU alone (within 85 % of each pod's maximum and above its minimum), it does *not* change the number of pods. CPU per pod is sized for the pods that are serving *now*, so the fast actuator covers the gap while new pods boot, and hands CPU back when they arrive.

Step 4: The regulator that remembers pods in flight

The outer loop's decision is made by an LQR controller. Its "state" (what it looks at) is:

$e = [\,\text{CPU per pod}-\text{target},\ \ \text{serving pods}-\text{target pods},\ \ \text{pods ordered last step but not ready yet}\,].$

The third number is the important one. Because the controller knows those pods are on their way, it does not order them again. With a 20 s boot time and a 15 s loop, one "in flight" slot is enough. The LQR gain K is computed once by solving the Riccati equation, and the Riccati solution also proves the loop is stable (it gives a Lyapunov function for free).

Step 5: The fast inner loop (every 5 seconds)

Only this loop changes CPU:

- **Raise at once** if the CPU needed is more than 30 % above the current allocation (the *dead-band*, which stops tiny back-and-forth changes), or if a statistical test shows a genuine surge in the last few seconds.
- **Give CPU back** once the need has stayed 15 % below the allocation for two checks in a row (10 seconds). Getting CPU back takes 0.1 s, so there is no reason to hold it "just in case".

A worked example

1. Quiet traffic: 2 pods with 0.5 cores each. Everything is calm.
2. A surge starts. Within 5 seconds the inner loop sees it and raises each pod to 0.9 cores, in place, in 0.1 s. No pod restarts. The users barely notice.
3. The surge keeps growing. Two pods at their maximum CPU are not enough, so the outer loop orders 2 more pods and records "2 in flight".
4. 15 seconds later the new pods are not ready yet. A normal autoscaler might order 2 more. CHIMERA sees "2 in flight" and waits.
5. The new pods become ready. CPU per pod is now more than needed, so the inner loop lowers it: the fast actuator hands the capacity back.
6. The surge ends. CPU drops within about 10 seconds; pods are removed later, more carefully.

What was removed, and why

The original design ("v1") also had a *coupling penalty* (punishing the two actuators for moving in opposite directions) and a *Lyapunov safety filter* (checking every action for stability). Tests showed both change **no decision at all**: once each actuator has a single owner, opposite moves do not happen, and the LQR is already proven stable. They were taken out of the final controller (v6). This is reported openly.

6 The competitors (baselines)

Baseline	What it does
HPA	Kubernetes standard. Adds/removes pods of fixed size (0.5 cores) to keep CPU near a target (70 %). Reactive.
VPA	Five fixed pods; changes their CPU size from usage.
HPA+VPA	Both standard tools running together on the same CPU signal.
AAPA	The paper this project started from (arXiv 2507.05653). Classifies traffic into four types (spike, periodic, ramp, stationary) and uses a tuned policy for each, with confidence. Pods only.
MagicScaler-style	Our implementation of the idea of MagicScaler (VLDB 2023): forecast with uncertainty, choose the number of pods trading cost against the risk of slow requests. Pods only.
KOSMOS	ICSOC 2021. Changes CPU every 5 s with a PI controller on response time, and pods every 300 s with rules. The closest existing system. Reimplemented from the paper.

All controllers get the same limits (CPU per pod 0.5–4 cores, 2–30 pods) and start from the same state.

7 How it was tested

7.1 The simulator

Running thousands of experiments on a real cluster is impossible, so most testing uses a **simulator** (built on the open-source AAPA simulator, extended to two dimensions). It models: requests arriving randomly, each needing on average 0.1 s of CPU; the slow-down when CPU gets busy ($\frac{1}{1-u}$); a new pod taking 20 s to boot; an in-place resize taking 0.1 s; and an SLO of 0.5 s. One run simulates 30 scored minutes of a trace (after a 20-minute unscored warm-up, so nobody is judged on its cold start).

7.2 The traffic (four sets of traces)

Set	What	Used for
Dev	5 Azure Functions traces (an earlier time window) and 4 synthetic shapes	designing
Held-out	the same 5 Azure traces (a later window) and 4 *different* synthetic shapes (flash crowd, daily cycle, step down, on/off)	judging
Serverless	10 real production functions from Huawei Cloud (2023, public, CC BY 4.0)	judging
Web	10 windows from NASA (1995) and ClarkNet (1995) web-server logs	judging

The rule: every design choice was made on the dev set only. The verdict comes from data no design decision saw. The serverless and web traces were chosen by a fixed rule written down *before* any controller ran on them. This prevents "tuning to the test".

7.3 The fair comparison: at equal CPU

Comparing each autoscaler at its default setting is misleading: one that spends more CPU will naturally have fewer slow requests. So every autoscaler is run at several settings (from generous to stingy), which gives a **cost–violation curve** (a "frontier") for each. Then, for each trace, we ask: *at exactly the CPU cost CHIMERA used, how many slow requests would the other autoscaler have?* (read off its curve). If CHIMERA's number is lower, CHIMERA wins that comparison.

7.4 Statistics

Each comparison is repeated with 5–10 seeds. A Wilcoxon signed-rank test checks whether the differences are consistently in one direction (not luck), and a Holm correction adjusts for testing several baselines at once. "$p < 0.001$" means the result is very unlikely to be chance.

7.5 Scale of the experiments

Held-out set: 3,870 main runs plus 4,095 sweep runs. Each external set: 900 + 3,150 runs. Robustness: 8 scenarios × 2,205 runs. Plus ablations, tuning runs and the live experiments: well over 30,000 simulated runs in total.

Part 3 Results

8 Where CHIMERA outperforms

8.1 At equal CPU, against everything

	Held-out		Serverless (Huawei)		Web (NASA, ClarkNet)	
Baseline	traces won	gap	traces won	gap	traces won	gap
AAPA	9/9	+3.30	10/10	+2.02	10/10	+2.00
HPA	9/9	+6.19	10/10	+3.99	10/10	+4.53
HPA+VPA	9/9	+5.16	10/10	+2.72	10/10	+3.89
KOSMOS	9/9	+5.81	10/10	+2.38	10/10	+1.27
MagicScaler-style	8/9	+5.87	10/10	+4.37	10/10	+4.66
VPA	6/6	+6.00	9/9	+1.77	10/10	+2.20

"Gap" = baseline's SLO violation rate minus CHIMERA's, in percentage points, at the same CPU cost. All $p < 0.001$.

Headline: 47 of 48 comparisons won on the held-out set, and **117 of 117** on the twenty real traces never used in design.

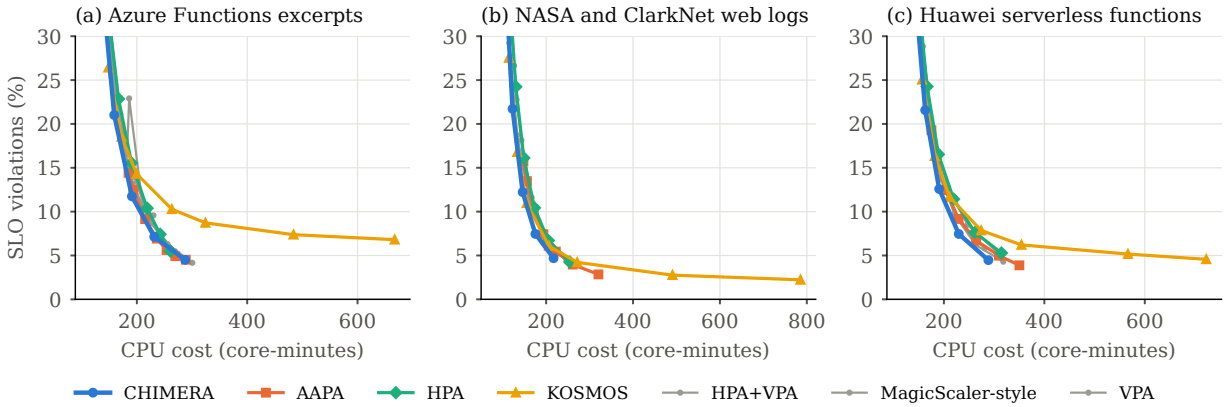


Figure 2. Cost (x) against slow requests (y) for each autoscaler at several settings. Down-left is better. CHIMERA's curve (blue) is the lowest.

8.2 At default settings

	Held-out, Azure traces		Serverless (Huawei)	
Autoscaler	SLO violations	CPU (core-min)	SLO violations	CPU (core-min)
CHIMERA	**5.93 %**	251.5	**6.06 %**	251
AAPA	6.92 %	236.8	9.18 %	230
KOSMOS	8.71 %	326.7	6.16 %	364
HPA+VPA	14.06 %	194.8	14.75 %	191
HPA	15.60 %	190.8	16.47 %	190
MagicScaler-style	16.78 %	181.1	19.21 %	179
VPA	19.40 %	169.2	21.85 %	169

p99 latency on the Azure traces: CHIMERA 0.82 s, AAPA 0.91 s, HPA 3.65 s, KOSMOS 3.87 s.

8.3 It still holds when the simulator's rules change

Because CHIMERA's risk pricing uses the same latency formula as the simulator, a fair critic asks: "Does it only win because it knows the simulator's rules?" We re-ran the held-out comparison with 8 different latency laws (different request-size distributions and different slow-down formulas), without re-tuning CHIMERA.

- In 6 of 8 scenarios CHIMERA still wins every comparison (42/42).
- With "bimodal" requests (10 % of requests are very slow), it wins 37–38 of 42 and loses narrowly to AAPA on four traces.
- The risk-pricing part specifically is worth about 1 point when its formula matches, and about 0 when it does not. The *structure* (fast loop, single owner, replica band, in-flight memory) carries the robust part of the win.

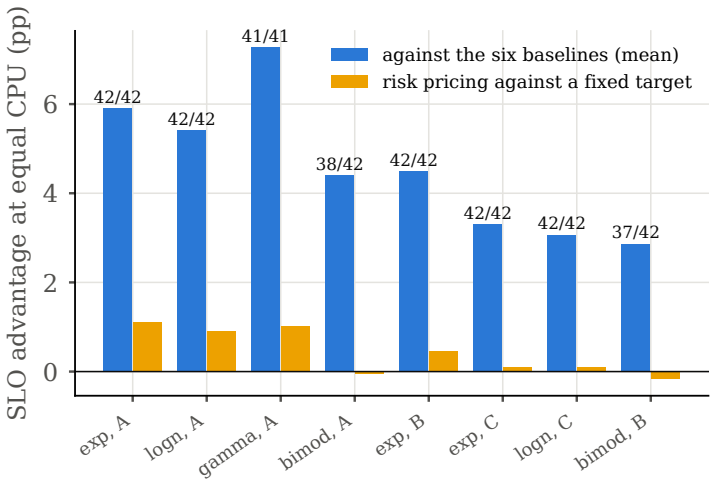


Figure 3. Blue: advantage over the six baselines under each latency law. Orange: what risk pricing alone adds.

8.4 Which parts matter (ablation)

Variant (held-out, Azure traces)	Violations	CPU	Resizes/run	Pod changes/run
Full CHIMERA	5.93 %	251.5	32.5	0.5
without fast release	2.24 %	480.1	1.0	0.5
pods only (no CPU resizing)	4.80 %	271.7	12.0	12.0
without the 5 s inner loop	4.92 %	274.6	37.2	0.4
without the 30 % dead-band	4.96 %	269.1	84.1	0.5
without the replica band	5.88 %	256.2	43.3	8.6
+ coupling penalty / + Lyapunov filter	5.93 %	251.5	32.5	0.5

How to read it: a row with fewer violations but much more CPU is just a more expensive setting, not better. Without fast release CPU nearly **doubles**. Pods-only needs 8 % more CPU here (60 % more on the synthetic shapes) and changes pods 24 times as often. The dead-band cuts resizes from 84 to 32 per run. The last row shows the two removed parts change nothing.

9 On a real cluster

9.1 It works on real Kubernetes

The same controller runs as a Kubernetes **operator** (a program inside the cluster). It reads CPU usage or request rate from **Prometheus** (a metrics system), resizes pods through the `pods/resize` API, and changes pod counts through the scale API. It was tested on a local cluster (`kind`, Kubernetes 1.33.1) and on an AWS server (`t3.small`, k3s 1.33). Checks passed on both: real metrics, CPU raised in place with **zero restarts**, pods added without flapping, CPU released after load, status written back.

9.2 Head-to-head with the Kubernetes HPA

A load generator replayed a 24-minute trace (real Azure traffic, then a spike to five times normal) against a real web service, measuring each request's latency. Both sides could use at most 10 cores. Each autoscaler ran at three settings, three seeds each.

HPA target	Slow requests	CPU	CHIMERA setting	Slow requests	CPU
70 %	48.9 %	85.3	0.46 (default)	32.1 %	80.9
50 %	23.5 %	102.4	0.35	14.6 %	93.6
35 %	16.3 %	124.3	0.25	**7.0 %**	108.1

Slow = over 0.3 s. CPU in core-minutes. Mean of 3 seeds.

- **7 of the 9 HPA runs were beaten outright** by a CHIMERA run with the same seed (less or equal CPU *and* fewer slow requests). **No CHIMERA run was beaten that way.**
- At equal cost CHIMERA had fewer slow requests in 5 of 6 comparable runs (mean 11.9 points better).
- CHIMERA at 0.25 had less than half the slow requests of the HPA's best setting, with 13 % less CPU.

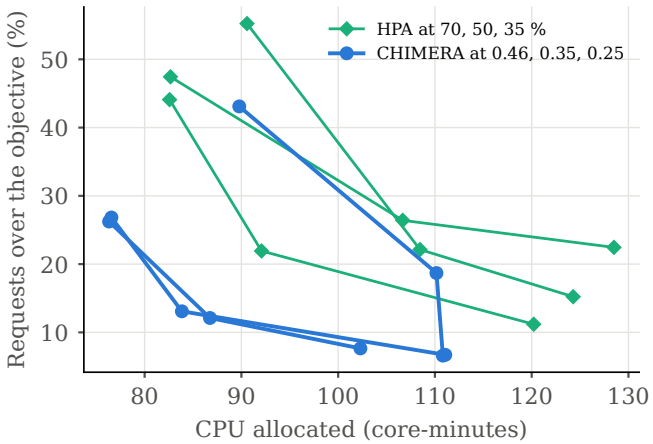


Figure 4. Real cluster. Each line is one seed. CHIMERA (blue) is below and left of the HPA (green).

Be honest about the real-cluster result

It is three seeds, one service, one trace, one laptop: *indicative*, not statistically proven. One CHIMERA run was repeated because the laptop interfered (the service was slow even with far more CPU than needed); both attempts are kept and reported. Every autoscaler had far more slow requests than in simulation, which shows the simulator is kinder than reality.

10 Where CHIMERA does *not* win

Know these before the viva

1. **KOSMOS at its default setting** has fewer slow requests than CHIMERA on three of four trace groups (for example 4.22 % vs 6.18 % on the web traces), but uses 43–84 % more CPU and about ten times as many resizes. CHIMERA wins only once cost is equal.
2. **The margins are modest**: about 2 points over AAPA and 1.3–2.4 over KOSMOS on the real traces.
3. **One held-out loss**: the MagicScaler-style scaler on one flat, noisy Azure trace, by 0.66 points.
4. **Bimodal requests**: loses narrowly to AAPA on 4 of 9 traces.
5. **Risk pricing** helps only when its latency formula is roughly right.
6. **More actions**: about 31 CPU resizes per half hour vs about 7 pod changes for AAPA. Each resize is cheap, but the load on Kubernetes at very large scale was not measured.
7. **The novelty is the combination**, not the individual ideas (see Section 11).

11 The story of the project: versions, mistakes and fixes

11.1 Versions

Version	What changed	Azure SLO viol.
v1	first design: LQR + coupling penalty + Lyapunov filter	11.82 %
v2	fixed in-flight counting, vertical bridging, conformal margin	7.06 %
v3	added the 5 s inner loop (cascade)	7.35 %
v4	inner loop owns CPU, replica band, drift reset	7.39 %
v5	risk-priced capacity	6.13 %
v6 (final)	removed inert parts; 30 % dead-band halved resizes	5.93 %

(All re-measured on the same held-out data. v3 and v4 look slightly worse at default settings because they spend less CPU; at equal CPU, v6 beats every earlier version: v1 by about 7.9 points, v2 by 1.9, v3 by 1.4, v4 by 1.1 and v5 by 0.3.)

11.2 Mistakes found and corrected (this is a strength, explain it as one)

- **Simulator bugs that affected everyone:** the load sensor could not report more than 6.7 requests per second; shared state leaked between runs. Fixed for all controllers.
- **Unfair baselines:** MagicScaler under-provisioned 3 times (wrong pod size); HPA+VPA's VPA part never fired. Fixed before comparing.
- **KOSMOS was mis-tuned.** Its settings were tuned before a simulator fix and made it oscillate. Re-tuned on dev data, its violations fell from 19.8 % to 8.7 %. Our claimed advantage over it fell from 19 points to about 6, and "26 times better tail latency" became 4.7 times. We published the corrected numbers.
- **An inflated claim:** "19 times better p99 than AAPA" was mostly AAPA starting cold; with a warm-up it is 1.1 times.
- **A discarded experiment:** the first web-trace run scaled traffic so high that the pod-only baselines hit their 30-pod limit; CHIMERA's "wins" there were unfair. We discarded it and

re-ran with fair scaling.

- **Real-cluster bugs the simulator could not show:** a pod that booted faster than expected was counted twice and removed 15 s after being added; four similar operator problems followed. Each was fixed and has an automatic test.
- **Fake references:** two papers in an early draft could not be found anywhere and were removed.

12 What is new, and what is not

Honest positioning, after reading the closest papers:

- **Not new:** resizing in place before adding pods (Themis, 2024); optimising pods and CPU together (Ship et al., IEEE CLOUD 2024); trading cost against risk (MagicScaler, 2023); conformal prediction in an autoscaler (BACC, 2026).
- **New in CHIMERA:** (1) one feedback controller that explicitly tracks pods still booting, for the pair of a 0.1 s and a 20 s actuator; (2) handing CPU back as pods arrive, plus a band in which pods do not move; (3) conformal "possible futures" used to price risk, in a rule that reduces to an ordinary target when the future is certain; (4) an evaluation that separates design data from judging data, compares at equal cost, and reports what did not work.

13 Limitations

1. Most numbers come from simulation; the real-cluster study is small.
2. Twenty real traces, 100 minutes each, from three sources.
3. The held-out set was used twice (v5, then v6); the v6 change was chosen on dev data by a rule written beforehand.
4. KOSMOS and MagicScaler are our implementations, not the authors' code.
5. The simulator's boot time and resize time are constants; reality varies.
6. Only CPU is scaled, not memory.
7. The CPU cost of a request must be given to the operator when it uses request rate.

Part 4 Viva preparation

14 Project map and how to run it

14.1 Important files

`src/core/chimera_controller.py`	the LQR controller (Riccati, in-flight state, lean path)
`src/core/chimera_simulator.py`	the 2-D cluster simulator and CHIMERA's loops
`src/scripts/run_full_evaluation.py`	runs every controller on every trace and seed
`k8s/operator_core.py`, `k8s/manager.py`	the real Kubernetes operator
`k8s/replay/`	the real-cluster head-to-head experiment
`tests/`	18 simulator tests, 32 operator tests
`results/REPORT_v6.md`	held-out results
`results/REPORT_hw.md`, `REPORT_ext.md`	serverless and web results
`results/replay3/REPLAY_REPORT.md`	real-cluster results
`paper/chimera.pdf`	the conference paper
`IMPROVEMENT_AUDIT.md`	every check, mistake and fix
`DEMO.md`, `demo/RUN_DEMO.cmd`	the lab demo

Code: `https://github.com/adithyan-css/CHIMERA_OS`

14.2 Live demo in two steps

1. Start Docker Desktop and wait for "Engine running".
2. Double-click `demo\RUN_DEMO.cmd` and follow the prompts. It checks the cluster, runs the simulation comparison, opens two windows (operator decisions, pods and CPU), adds load (watch the CPU of the same pods rise with zero restarts), removes it (watch the CPU come back), and opens the paper.

15 Viva questions and answers

The basics

Q. What is CHIMERA in one sentence?

Answer. A Kubernetes autoscaler that controls both the number of pods and the CPU of each pod with one coordinated decision, using the fast actuator (in-place resize) first and remembering pods that are still starting.

Q. What problem does it solve?

Answer. Keeping requests fast at the lowest CPU cost when traffic changes, now that CPU can be resized in 0.1 s while new pods still take tens of seconds.

Q. What is horizontal vs vertical scaling?

Answer. Horizontal: more pods. Vertical: more CPU per pod. Horizontal is slow (boot time), vertical is fast since in-place resize.

Q. What is in-place pod resize, and why does it matter?

Answer. Kubernetes feature (KEP-1287, on by default from 1.33) that changes a running pod's CPU without restarting it. Without it, vertical scaling would need a restart and could not be used for fast reactions; with it removed, CHIMERA's violations on the synthetic traces rise from 5.63 % to 7.91 %.

Q. What is an SLO and what is an SLO violation?

Answer. A latency promise, here 0.5 s per request in simulation (0.3 s on the real service). A violation is a request slower than that. We report the percentage of such requests.

Q. Why not just give the application lots of CPU?

Answer. Cost. Every allocated core is paid for. The goal is the best latency for a given cost, which is why we compare at equal CPU.

Q. What is a core-minute?

Answer. One core allocated for one minute. Our cost measure.

How it works

Q. What are the two loops?

Answer. Outer, every 15 s: decides total capacity and the number of pods (LQR). Inner, every 5 s: changes CPU per pod only (raise past a 30 % dead-band or on a surge; release after two quiet checks).

Q. Why two loops at different speeds?

Answer. Each actuator gets a loop matching its speed and a single owner, so they never fight. The ablation shows CPU nearly doubles without fast release, and resizes rise from 32 to 80 if both loops can change CPU.

Q. What is LQR and why use it?

Answer. Linear-quadratic regulator: an optimal feedback rule $u = -Ke$ balancing tracking error against action size. It gives a principled joint decision and, through the Riccati solution, a stability proof.

Q. What is the "in-flight" state?

Answer. The pods ordered last step that are not ready yet. Including them in the state stops the controller ordering the same capacity twice. On the real cluster we reconcile it with what Kubernetes reports (desired minus ready pods).

Q. Why one in-flight slot?

Answer. Boot time 20 s, loop 15 s: an order placed now is ready before the step after next, so it is pending for exactly one step.

Q. How do you prove stability?

Answer. The Riccati solution P is positive definite and $V = e^\top Pe$ decreases under $u = -Ke$, so V is a Lyapunov function. That is why the separate Lyapunov filter never acted and was removed.

Q. What is risk-priced capacity?

Answer. Choose total cores minimising expected slow requests over several plausible futures plus a price per core. With a certain future it reduces to demand / target utilisation.

Q. What is conformal prediction and why use it?

Answer. A method that makes a forecaster's uncertainty range honest: it uses recent errors to size the range so it covers the truth about 90 % of the time, without assuming a distribution. We use it to build the possible futures.

Q. What is the replica band?

Answer. A range of pod counts within which the needed capacity can be reached by resizing CPU alone. Inside it, pods are not added or removed, because each new pod wastes its boot time. It cuts pod changes from 8.6 to 0.5 per run.

Q. What is the dead-band and why 30 %?

Answer. CPU is raised only if the need exceeds the allocation by more than 30 %. It stops tiny back-and-forth resizes. 30 % was the widest value that did not hurt efficiency on the dev data; it halved resizes (62 to 31 per run).

Q. What is ADWIN?

Answer. A drift detector: it notices when the traffic pattern changes, so the safety margin forgets the old pattern.

Q. Why did you remove the coupling penalty and the Lyapunov filter?

Answer. Tests showed they changed no decision in any of 90 runs. Single ownership prevents opposite moves, and the LQR is already stable. Removing them changed no result.

Testing

Q. Why a simulator?

Answer. Tens of thousands of controlled, repeatable experiments with identical traffic for every controller. A real cluster cannot do that in reasonable time. We then checked on a real cluster.

Q. How do you avoid tuning to the test data?

Answer. All choices on a dev split; verdict on held-out data, plus twenty real traces chosen by a rule written before any run.

Q. What does "at equal CPU" mean?

Answer. Each controller is run at several settings to get a cost–violation curve; we read each baseline's violation rate at exactly CHIMERA's CPU cost. Otherwise a controller could look better just by spending more.

Q. What statistical test did you use?

Answer. Wilcoxon signed-rank on paired gaps (same trace, same seed), Holm-corrected for multiple baselines. All main comparisons $p < 0.001$.

Q. Why a 20-minute warm-up?

Answer. So no controller is judged on its cold start. Without it, AAPA looked 19 times worse in p99 than it really is.

Q. What are the traces?

Answer. Five Azure Functions excerpts, ten Huawei Cloud serverless functions (2023), ten NASA and ClarkNet web-log windows (1995), plus synthetic shapes (flash crowd, daily cycle, step down, on/off).

Results

Q. Does it outperform the others?

Answer. At equal CPU, yes: 47 of 48 comparisons on the held-out set and 117 of 117 on the twenty real traces, all significant. On a real cluster, 7 of 9 HPA runs were beaten outright and no CHIMERA run was.

Q. By how much?

Answer. Modestly: about 2 percentage points fewer slow requests than AAPA at equal CPU on the real traces, 1.3–2.4 than KOSMOS, 4–5 than HPA.

Q. Which baseline is strongest?

Answer. KOSMOS, after re-tuning. At its default setting it matches or beats CHIMERA's violation rate on three of four trace groups but spends 43–84 % more CPU.

Q. Where does CHIMERA lose?

Answer. One held-out trace vs the MagicScaler-style scaler (0.66 points); four traces vs AAPA when requests are bimodal; KOSMOS at default settings if you ignore cost.

Q. Which part gives the most benefit?

Answer. Fast release of CPU, then joint control (using CPU resizing at all), then single ownership and the dead-band. Risk pricing adds about one point and depends on its model.

Q. How did it do on the real cluster?

Answer. Against the Kubernetes HPA with three settings each and three seeds: CHIMERA at 0.25 had 7.0 % slow requests at 108 core-minutes vs the HPA's best 16.3 % at 124; 7 of 9 HPA runs beaten outright, none the other way. Zero container restarts in all runs.

Q. Why were violation rates so high on the real cluster?

Answer. A 61 ms request on a pod limited to half a core exceeds one CPU-quota period, so the real service queues much more than the simulator's formula assumes. It applies to every autoscaler equally; it shows the simulator is optimistic.

Critical questions

Q. Isn't this only simulation?

Answer. Most numbers are, but the operator runs on real Kubernetes (local and AWS), and a real-cluster comparison with measured latency points the same way. That study is small and we say so.

Q. Did you tune CHIMERA more than the baselines?

Answer. No more: KOSMOS was re-tuned on dev data like CHIMERA, baselines had bugs fixed before comparison, and every controller is compared along its whole cost curve.

Q. Your risk model matches the simulator. Isn't that cheating?

Answer. We tested exactly that: 8 other latency laws without re-tuning. The win over baselines survives; only the risk-pricing part's small gain disappears. We report this.

Q. What is genuinely new?

Answer. The combination: a feedback controller tracking pods in flight for a fast/slow actuator pair, handing CPU back as pods arrive, a replica band, and conformal futures for risk pricing, plus a careful evaluation. The individual ideas exist in Themis, Ship et al., MagicScaler and BACC.

Q. Why so many CPU resizes?

Answer. About 31 per half hour after the dead-band (down from 62). Resizes are cheap (0.1 s, no restart). KOSMOS makes about 250. Fleet-scale API load is untested.

Q. What would you do next?

Answer. A larger real-cluster study (several nodes, a multi-service app, more seeds, real AAPA and KOSMOS operators); learn the latency curve from measurements; scale memory too.

Q. Why is it called CHIMERA?

Answer. A chimera combines parts of different creatures; the controller combines horizontal and vertical scaling in one body.

Q. What tools and languages?

Answer. Python (NumPy, SciPy, LightGBM, SimPy, pandas), Kubernetes 1.33 (kind locally, k3s on AWS EC2), Prometheus, Docker, LaTeX for the paper.

Q. How long does one control step take?

Answer. Well under a millisecond (about 0.3–0.5 ms on a laptop), against a 5–15 s loop.

Q. How do you know the operator is correct?

Answer. 32 automatic tests (including regression tests for each live bug), five live checks passing on two clusters, and the head-to-head replay.

16 Glossary

Term	Meaning
Ablation	Removing one part to measure its effect.
ADWIN	Adaptive windowing: detects a change in a data stream.
Baseline	Existing method compared against.
Cascade	Two nested control loops: a fast inner one and a slower outer one.
Cluster	A group of machines run together by Kubernetes.
Conformal prediction	Distribution-free way to make prediction intervals with guaranteed coverage.
Container	A packaged program that runs the same way anywhere.
Core-minute	One CPU core allocated for one minute; cost unit.
CQR	Conformalized quantile regression.
Dead-band	A minimum change below which no action is taken.
Dev / held-out split	Data for designing vs data for judging.
Equal-CPU comparison	Comparing violation rates at the same CPU cost.
Frontier	Curve of cost vs violations as a controller's setting changes.
HPA / VPA	Kubernetes horizontal / vertical pod autoscalers.
In-flight pods	Pods ordered but not yet ready.
In-place resize	Changing a running pod's CPU without restart (KEP-1287).
kind, k3s	Small Kubernetes distributions (local in Docker; lightweight server).
LQR	Linear-quadratic regulator: optimal linear feedback control.
Lyapunov function	An "energy" that always decreases in a stable system.
Operator	A program inside Kubernetes that manages something automatically.
p99	99th-percentile latency.
Pod / replica	One running copy of the application / an additional copy.
Prometheus	Monitoring system that collects metrics.

Term	Meaning
Replica band	Range of pod counts in which CHIMERA keeps the count fixed.
Riccati equation	The equation whose solution gives the LQR gain and stability proof.
Seed	Starting value for random choices; same seed, same traffic.
SLO	Service level objective: the latency target.
Trace	A recording of real traffic over time.
Utilisation	Demand divided by capacity.
Wilcoxon test	Statistical test for consistent paired differences.

17 One page of numbers to remember

Memorise this

- Fast actuator: in-place resize, **0.1 s**. Slow actuator: new pod, **20 s** boot (simulator).
- Loops: outer **15 s** (pods), inner **5 s** (CPU). Dead-band **30 %**; release after **2** checks **15 %** below. Default setting **0.46**.
- SLO: **0.5 s** (simulator), **0.3 s** (real service, 61 ms per request).
- Held-out: **47/48** wins at equal CPU. Real traces never used in design: **117/117** (57 serverless + 60 web). All $p < 0.001$.
- Advantage at equal CPU on real traces: about **+2.0** points over AAPA, **+1.3 to +2.4** over KOSMOS.
- Defaults, Azure held-out: CHIMERA **5.93 %**, AAPA 6.92 %, HPA 15.60 %.
- Robustness: wins all comparisons in **6 of 8** latency laws.
- Ablation: no fast release ⇒ CPU ×**1.9**; replica band cuts pod changes **8.6** → **0.5**; dead-band cuts resizes **84** → **32**.
- Resizes per half hour: **about 31** (v5: 62; KOSMOS: about 250).
- Real cluster vs HPA: **7 of 9** HPA runs beaten outright, **0** CHIMERA runs beaten; CHIMERA 0.25: **7.0 %** at 108 core-min vs HPA best 16.3 % at 124.
- KOSMOS re-tune: 19.8 % → 8.7 % violations; our advantage 19 → 6 points.
- Tests: **18** simulator + **32** operator. Control step: **under 1 ms**.
- Versions: v1 11.82 % → v6 5.93 % (Azure held-out).